

Review Cheat Sheet: RUDP and Smart-RUDP

Two pages to read before you go in. 9 Oct 2026.

Why this is a project, not a lab exercise

A lab follows a given specification. We chose a problem (build TCP's guarantees on UDP), designed several cooperating parts (13-byte header, handshake, Go-Back-N, cumulative ACKs, fast retransmit, AIMD, loss simulation), measured them (benchmark sweep, window graphs, Wireshark captures with our own dissector, Smart-RUDP vs AIMD), documented what broke and how we fixed it, and used the result in a routed network design and a working chat between two devices. We also report a negative result: Smart-RUDP did not beat AIMD.

One line: a lab exercise sends a datagram; we built the reliability and congestion control that sit on top of it, measured them, and used them in a real application.

Numbers to remember

Item	Value
Header	13 bytes: seq 4, ack 4, flags 1, checksum 2, length 2
Data packet on the wire	53 bytes (ACK or handshake: 13)
Timeout / fast retransmit	500 ms / 3 duplicate ACKs
Window	slow start to 16, then +1 per RTT, halve on dup ACKs, reset to 1 on timeout, cap 32
Benchmark	62 of 62 packets delivered from 0 to 40% loss
Smart-RUDP	Q-learning, 80 states, 5 actions, 40 training episodes, 6 eval runs per path
Smart-RUDP result	Experiments: +63% time on 5G-like, +10% satellite, tie on lossy. Chat (simulated): faster than AIMD on 5 of 6 held-out paths, equal on 1
Network	4 routers, OSPF area 0, 2 VLANs, DHCP, ACL; server LAN metric 2 via R3, branch metric 3 via R2
Failover	shut R1 Gi0/2: 70 of 71 pings, rerouted via R2
Chat	window 8, 300 ms timeout; terminal and browser; laptop and phone over Tailscale

Hard questions, short answers

Why not TCP? TCP is in the kernel and cannot be changed. Building our own makes every mechanism visible. QUIC and HTTP/3 do the same.

Stress test? 48 unseen random paths x 3 seeds (loss to 30%, jitter, bottlenecks): mean 3.73 s Smart, 3.88 s fixed, 7.16 s AIMD. Smart beat AIMD on 27, lost on 5, within 5% on 16. Weak: tight queues, very high loss. A broader-trained agent was worse, so we kept the first. Live: 200 msgs at 30% loss both ways arrived in order in 4.3 s.

Deep RL? Yes, in the chat: `web/dqn.py`, a numpy DQN (5-32-32-5, replay buffer, target network), trained in the simulator. Mean 3.20 s vs table 3.73 s vs AIMD 7.16 s on the 48 unseen paths; faster than the table on 19, slower on 6. Seed-sensitive (1 of 4 seeds failed), so we pick by validation. Python only, simulated; the C agent is still tabular.

TCP and UDP baselines? Simulated on the same 48 paths: plain UDP delivers 68% (all messages on 4 of 48 paths). TCP model (selective ACK): mean 4.56 s, median 1.48 s, the best median. Deep Smart-RUDP 3.20 s mean, about even with it path by path (22 faster, 23 slower). TCP row is a model, not kernel TCP; real netem script provided, not run here.

Why is RUDP slower than TCP? Go-Back-N resends the whole window. A selective-ACK variant in the simulator cut AIMD from 7.16 s to 4.90 s (median 2.72 to 1.44). First item of future work.

Reordering? Tested in the simulator only: AIMD with Go-Back-N goes 0.82 s to 6.79 s at 20 ms jitter; learned agents cope better; SACK removes most of the damage.

How sure are the C results? 20 runs per mode, 95% intervals: 5G-like Smart +0.92 s (slower), satellite-like +6.3 s (slower), lossy -0.37 s (tie).

Does Smart-RUDP beat AIMD now? The neural agent does, in the C transport on the three emulated paths: 1.18 vs 1.38 s (5G-like), 6.63 vs 9.13 s (lossy), 13.1 vs 17.9 s (satellite-like), 95% intervals exclude zero. The table agent does not (slower on 5G-like and satellite-like). Caveats: trained in a simulator, same start window 10, AIMD cap 128 vs agents 32, more retransmissions on random loss.

SACK in C? Built (`make sack`, `--sack 1`): about 10x fewer retransmissions, no significant change in completion time on these paths. In the simulator it gave large gains; those paths differ.

How do you know it works? `make test`: wire format, end to end at 0% and 20% loss, every controller mode with and without SACK, simulator behaviour.

Did Smart-RUDP win? Depends on the setting. Main experiments (5G, lossy, satellite): tied on one path, slower on two (20-run intervals confirm), and we report that. Chat, retrained on simulated bursty random-loss paths: faster than AIMD on 5 of 6 held-out paths, equal on 1, because AIMD resets to 1 on every timeout. Both start from window 10; results are simulated; the agent may not shrink the window without a congestion signal (without that rule it lost).

Why Go-Back-N? Simpler to build and explain. It wastes bandwidth under loss; Selective Repeat with SACK is future work.

Tests ran on one machine? Yes, with emulated delay, loss and jitter. They do not prove internet performance and the report says so. The laptop-to-phone chat is a small real-network check.

What does Packet Tracer prove? The network design (OSPF, VLANs, ACL, failover). It cannot run C, so the protocol is validated separately with emulation and Wireshark.

What if the peer disappears? The sender keeps retransmitting on timeout. A connection abort after repeated timeouts is not built; it is the fix we would add.

Does the checksum catch errors? 16-bit one's-complement sum like UDP and TCP; a corrupted packet fails and is retransmitted. It cannot catch every error pattern.

Is the loss real? Benchmark and chat loss is simulated in the program. We also captured a real loopback run with tshark and netem loss.

Why no handshake in the exported trace? The export builds synthetic packets. The real tshark capture shows SYN, SYN-ACK, ACK.

Is the Q-learning really learning? Yes: the learning-curve figure shows it. It is small and tabular with limited training.

Who did what? Agree one answer as a team before you go in.

Say the limits first

Go-Back-N was replaced as the default by **Selective Repeat** (buffering receiver + ACK bitmap, per-packet timers); the older tables were measured with Go-Back-N. Protocol tests are loopback only. Smart-RUDP did not beat AIMD. The agent is small. **Next:** adaptive timeout in C, fairness in C, real-network tests.

If asked: is it Selective Repeat now?

Yes, by default in C, the terminal chat and the web chat: the receiver buffers out-of-order packets (64 window) and reports them in a bitmap in every ACK; the sender has one timer per packet and resends only what is missing. Not textbook in one way: cumulative ACK plus bitmap, not one ACK per packet. `--gbn` / `RUDP_GBN` restores Go-Back-N. With it the neural agent beats AIMD by 9%, 46%, 50% (C, 10 runs) and the 10%-loss failure disappears (tie).

If asked: where does the agent fail?

At **10% loss** the plain neural agent took **85 s vs 11 s for AIMD** (C transport). Fix: **hybrid** (`--hybrid 0.10`): AIMD drives while a decision interval loses 10% or more; the agent resumes after 3 clean intervals. Result: benign paths keep the win, hostile path 13.9 s (not significantly different from AIMD). A timeout cut was tried and dropped. It is a rule we added, not something the network learned.

If asked: is the win real?

Our first simulator allowed a **dup-ACK retransmission storm**; with the realistic rule the simulated gain nearly vanishes (stress mean: AIMD 7.75 s, neural 7.27 s). The evidence we rely on is the **C transport with equal window caps (AIMD capped at 32): neural still wins by 19%, 29%, 28%, all significant** (10 paired runs). Three emulated paths, not real networks. A retrained agent in the realistic simulator was stable but much worse in C, so not used.

If asked: is it fair? does every part matter?

Fair against itself and on short paths; **74% share against AIMD on the 120 ms path** (Jain 0.82), simulator only. Ablation: only the **no-shrink mask** matters (6.54 s without, about 3.3 s with, AIMD 7.16 s). Training is **seed-sensitive** (3.2 to 7.4 s), so we pick the seed on validation paths.